

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments.(BL2, BL3, BL6)

Assessment Tool Weightage: 35%

Dr DUMPALA PRASANTH

QUESTIONS: 10

Total Marks: 50

Annexure A

Experiments

Duration: 3hrs

1. **Design and develop a Policy Gradient-based Reinforcement Learning** model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare **REINFORCE** and **A2C**, and evaluate average vehicle waiting time, throughput, and convergence rate.

(10 Marks)

2. **Design and implement an Actor-Critic (A2C/A3C)** model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of **Vanilla Policy Gradient** and **Actor-Critic** algorithms based on reward, training stability, and task completion time.

(10 Marks)

3. **Develop a Deep Deterministic Policy Gradient (DDPG)** model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance.

(10 Marks)

4. **Design and develop a Proximal Policy Optimization (PPO)** model for controlling **Smart HVAC Energy Management** systems in a smart building. Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.
(10 Marks)
5. **Implement a Trust Region Policy Optimization (TRPO)** algorithm for controlling a **Industrial** robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency.
(10 Marks)
6. **Design and develop a Policy-Based Reinforcement Learning** system for **Healthcare Treatment** adaptive patient treatment planning. Compare **A2C** and **PPO** in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.
(10 Marks)
7. **Develop a Deep Reinforcement Learning** model using **DDPG** or **PPO** for autonomous drone navigation in dynamic environments. Evaluate navigation accuracy, obstacle avoidance, energy consumption, and policy convergence.
(10 Marks)
8. **Design and implement an A3C-based Reinforcement Learning** framework for dynamic cloud resource allocation. Compare the algorithm with Vanilla Policy Gradient using metrics such as response time, CPU utilization, and resource efficiency.
(10 Marks)
9. **Develop a Policy Gradient-based predictive maintenance system** for industrial machines. Design the RL environment, reward function, and policy network to minimize maintenance costs and machine downtime. Compare the performance of **REINFORCE** and **PPO**.
(10 Marks)
10. **Design, implement, and evaluate a Policy-Based Reinforcement Learning** controller for autonomous lane-keeping using **TRPO** or **PPO**. Compare the selected algorithm with **A2C** based on lane deviation, safety, reward convergence, and training efficiency.
(10 Marks)

Code of Conduct:

I B.VANI(192472348) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B.VANI(192472348)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

EXPERIMENT 1: Policy Gradient-Based Traffic Signal Control

Aim: To design a Policy Gradient-based RL model for intelligent traffic signal control and compare REINFORCE and A2C.

State Space: Number of vehicles in each lane, current signal phase, queue length, waiting time, and traffic density.

Action Space: Select the next traffic-light phase or maintain the current phase.

Reward Function:

$$[R=-(\text{waiting time}+\text{queue length})]$$

A positive reward can be given for improved traffic flow and reduced congestion.

Method: REINFORCE directly updates the policy using the return:

$$[\nabla J(\theta)=E[\nabla \log \pi_{\theta}(a|s)G_t]]$$

A2C uses an actor to select actions and a critic to estimate the value function:

$$[A(s,a)=Q(s,a)-V(s)]$$

Implementation: Python with TensorFlow/PyTorch can be used to construct the policy network and value network. The environment records traffic states and returns rewards after each signal decision.

Evaluation: Average vehicle waiting time, traffic throughput, cumulative reward, and convergence rate are measured.

Expected Result: A2C is expected to provide more stable learning than REINFORCE because the critic reduces the variance of policy updates. The final policy should reduce waiting time and improve vehicle throughput.

EXPERIMENT 2: Actor-Critic Warehouse Robot

Aim: To implement an Actor-Critic model for an autonomous warehouse robot that collects and delivers packages efficiently.

State Space: Robot position, package locations, destination locations, obstacles, battery level, and current delivery status.

Action Space: Move up, down, left, right, collect a package, or deliver a package.

Reward Function: Positive reward for successful collection and delivery; negative rewards for collisions, unnecessary movement, delay, and excessive battery consumption.

Algorithms: Vanilla Policy Gradient directly updates the policy using observed returns. Actor-Critic uses an actor for action selection and a critic for estimating state value.

Evaluation: Cumulative reward, training stability, number of successfully delivered packages, and task completion time are measured.

Expected Result: Actor-Critic should generally provide faster and more stable learning because the critic supplies a lower-variance learning signal.

Conclusion: Actor-Critic is suitable for warehouse navigation because it can continuously improve the robot's policy while considering long-term rewards.

EXPERIMENT 3: DDPG for Continuous Portfolio Optimization

Aim: To develop a DDPG model for continuous portfolio optimization in stock trading.

State Space: Stock prices, historical returns, market indicators, current portfolio allocation, cash balance, and volatility.

Action Space: Continuous portfolio weights assigned to different assets.

Reward Function:

$[R_t = \text{Portfolio Return} - \lambda(\text{Risk}) - \text{Transaction Cost}]$

where (λ) controls the importance of risk.

DDPG Components: DDPG uses an **Actor network** to generate continuous portfolio allocations and a **Critic network** to estimate the value of the selected action.

Evaluation: Cumulative return, portfolio risk, Sharpe ratio, maximum drawdown, and convergence rate.

Expected Result: The trained policy should learn portfolio allocations that attempt to maximize long-term return while controlling risk and transaction costs.

Conclusion: DDPG is appropriate for continuous portfolio allocation because its action space is continuous rather than limited to buy, sell, and hold.

EXPERIMENT 4: PPO for Smart HVAC Energy Management

Aim: To implement PPO for controlling HVAC systems while minimizing energy consumption and maintaining occupant comfort.

State Space: Indoor temperature, outdoor temperature, humidity, occupancy, current HVAC setting, and energy consumption.

Action Space: Increase, decrease, or maintain HVAC temperature and operating level.

Reward Function:

$[R = -(\text{Energy Consumption}) + \lambda|\text{Indoor Temperature} - \text{Desired Temperature}|]$

PPO Method: PPO updates the policy using a clipped objective to prevent excessively large policy changes:

$[L^{\text{CLIP}}(\theta) = E[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]]$

Comparison: REINFORCE directly uses episode returns and may have high variance, while PPO provides more controlled and stable policy updates.

Evaluation: Energy consumption, temperature deviation, cumulative reward, occupant comfort, and convergence.

Expected Result: PPO should maintain comfortable indoor conditions while reducing unnecessary HVAC energy consumption and providing more stable training than REINFORCE.

EXPERIMENT 5: TRPO for Industrial Robotic Arm Control

Aim: To implement TRPO for controlling an industrial robotic arm during automated manufacturing.

State Space: Joint positions, joint velocities, target position, object position, and current assembly stage.

Action Space: Continuous changes to joint movements and motor controls.

Reward Function: Positive reward for reaching the target accurately and completing assembly; penalties for excessive movement, collisions, positioning errors, and time consumption.

TRPO: Trust Region Policy Optimization restricts the policy update so that the new policy does not move too far from the previous policy. This improves policy stability.

Evaluation: Policy stability, positioning precision, convergence speed, task completion time, and production efficiency.

Expected Result: TRPO should provide stable policy updates and accurate robotic movements while reducing unsafe or excessively large policy changes.

Conclusion: TRPO is suitable for industrial robotic control where stable and reliable policy improvement is important.

EXPERIMENT 6: Policy-Based RL for Adaptive Healthcare Treatment Planning

Aim: To design a policy-based RL system for adaptive patient treatment planning and compare A2C with PPO using simulated patient data.

State Space: Simulated patient age group, symptoms, vital measurements, previous treatment, treatment response, and health condition.

Action Space: Select an appropriate treatment strategy, adjust treatment intensity, continue treatment, or request further monitoring within predefined safe limits.

Reward Function: Positive reward for improvement in simulated health indicators and negative reward for adverse responses, ineffective treatment, or unsafe states.

A2C: Uses an actor to select treatment actions and a critic to estimate the state value.

PPO: Uses clipped policy updates to maintain stable policy improvement.

Evaluation: Treatment effectiveness, cumulative reward, learning stability, and convergence rate.

Expected Result: PPO is expected to provide more stable policy updates, while A2C can provide efficient learning through its actor-critic architecture.

Conclusion: Policy-based RL can be evaluated using simulated patient environments, but real clinical deployment would require extensive medical validation, safety constraints, and professional supervision.

EXPERIMENT 7: DDPG/PPO for Autonomous Drone Navigation

Aim: To develop a deep RL model for autonomous drone navigation in dynamic environments.

State Space: Drone position, velocity, battery level, obstacle positions, destination, weather conditions, and surrounding environment information.

Action Space: Continuous changes in speed, direction, altitude, and flight angle.

Reward Function:

$[R = R_{\text{progress}} - R_{\text{collision}} - R_{\text{energy}} - R_{\text{delay}}]$

A large positive reward is provided for reaching the destination.

Algorithm: DDPG can be used when continuous control is required. PPO can also be used for stable policy optimization.

Evaluation: Navigation accuracy, obstacle avoidance rate, energy consumption, successful delivery rate, and policy convergence.

Expected Result: The drone should learn routes that reach the destination while avoiding obstacles and minimizing unnecessary energy consumption.

Conclusion: Deep RL enables the drone to adapt its navigation policy to dynamic environmental conditions.

EXPERIMENT 8: A3C for Dynamic Cloud Resource Allocation

Aim: To implement an A3C-based RL framework for dynamic allocation of cloud computing resources.

State Space: CPU utilization, memory utilization, number of active requests, network traffic, response time, and available resources.

Action Space: Increase or decrease CPU/memory allocation, migrate workloads, or maintain current allocation.

Reward Function:

$[R = -(\text{Response Time} + \text{Resource Cost} + \text{Overload Penalty})]$

A3C: A3C uses multiple agents/workers that interact with environments in parallel. Each worker updates a shared policy, allowing diverse experiences to improve learning.

Comparison: Vanilla Policy Gradient generally learns from collected trajectories, whereas A3C benefits from parallel workers and can provide faster and more stable learning.

Evaluation: Response time, CPU utilization, resource efficiency, throughput, and cumulative reward.

Expected Result: A3C should allocate resources dynamically, reduce response time, and improve resource utilization under changing workloads.

EXPERIMENT 9: Policy Gradient-Based Predictive Maintenance

Aim: To develop a policy-gradient RL system for predictive maintenance that minimizes maintenance cost and machine downtime.

State Space: Machine temperature, vibration, pressure, operating hours, sensor readings, machine health condition, and previous maintenance history.

Action Space: Continue operation, inspect machine, perform preventive maintenance, or replace a component.

Reward Function:

$[R = -(\text{Maintenance Cost} + \text{Downtime Cost} + \text{Failure Cost})]$

A positive reward can be provided for maintaining healthy operation without unnecessary maintenance.

REINFORCE: Learns the policy directly from the accumulated return.

PPO: Uses clipped policy updates to improve stability and prevent excessively large policy changes.

Evaluation: Maintenance cost, machine downtime, number of failures, cumulative reward, and convergence rate.

Expected Result: PPO is expected to provide more stable training than REINFORCE and learn a policy that balances preventive maintenance against unnecessary maintenance.

Conclusion: Policy-based RL can optimize maintenance decisions by learning from machine operating conditions and maintenance outcomes.

EXPERIMENT 10: Policy-Based RL for Autonomous Lane-Keeping

Aim: To design and evaluate a policy-based RL controller for autonomous lane-keeping using PPO or TRPO and compare it with A2C.

State Space: Vehicle position relative to the lane center, vehicle speed, steering angle, lane curvature, heading error, and lateral deviation.

Action Space: Continuous steering control or discrete steering adjustments.

Reward Function:

$[R = -(\text{Lane Deviation} + \text{Heading Error} + \text{Unsafe Action Penalty})]$

A positive reward is provided for maintaining the vehicle near the lane center.

Algorithm: PPO/TRPO learns a stable policy for steering control, while A2C uses an actor and critic to update the policy.

Evaluation: Lane deviation, safety rate, cumulative reward, convergence rate, training time, and computational efficiency.

Expected Result: PPO or TRPO should maintain smaller lane deviations and provide stable policy convergence. A2C may learn efficiently but can be more sensitive to policy-update variations.

Conclusion: Policy-based RL can learn adaptive lane-keeping behavior by continuously interacting with a simulated driving environment and optimizing long-term safety and lane stability.

Overall Conclusion

The experiments demonstrate the application of policy-based reinforcement learning methods to traffic control, robotics, finance, smart buildings, healthcare, drones, cloud computing, predictive maintenance, and autonomous vehicles. Algorithms such as **REINFORCE**, **A2C**, **A3C**, **DDPG**, **PPO**, and **TRPO** provide different approaches for learning effective policies. The performance of each system can be evaluated using cumulative reward, convergence, efficiency, stability, task-specific performance, and resource utilization.

